

/*

=====

SIGE - Sistema Inteligente de Gestão de Energia
Arduino UNO - Monitoramento de Corrente e Tensão

=====

Mitigações implementadas:

- Remoção automática de offset
- Cálculo RMS
- Filtro de ruído
- Média por amostragem
- Comunicação serial estruturada
- Validação de faixa de leitura

Hardware:

- ACS712 30A
 - Sensor de tensão AC
 - Arduino UNO
- =====

*/

const int pinCorrente = A0;
const int pinTensao = A1;

// ----- CONFIGURAÇÕES -----

float vRef = 5.0;
int adcMax = 1023;

// ACS712 30A
float sensibilidade = 0.066;

// Calibrar depois com multímetro
float fatorCalibracaoTensao = 250.0;

// Filtro de ruído
float ruidoMinimo = 0.02;

// Offset automático
float offsetCorrente = 2.5;

// Intervalo de envio

```

unsigned long intervalo = 1000;
unsigned long ultimoEnvio = 0;

// Energia acumulada
float energia_kWh = 0;
unsigned long tempoAnterior = 0;

void setup() {

    Serial.begin(9600);

    delay(2000);

    // ===== CALIBRA OFFSET =====
    offsetCorrente = calibrarOffset();

    tempoAnterior = millis();

    Serial.println("SIGE_INICIADO");
}

// =====
// CALIBRA OFFSET DO ACS712
// =====
float calibrarOffset() {

    int amostras = 1000;
    float soma = 0;

    for (int i = 0; i < amostras; i++) {

        soma += analogRead(pinCorrente);

        delay(2);
    }

    float media = soma / amostras;

    return media * (vRef / adcMax);
}

```

```

// =====
// MEDIÇÃO RMS DA CORRENTE
// =====
float medirCorrenteRMS() {

    int amostras = 500;

    float soma = 0;

    for (int i = 0; i < amostras; i++) {

        float leitura =
            analogRead(pinCorrente) * (vRef / adcMax);

        float corrente =
            (leitura - offsetCorrente) / sensibilidade;

        // FILTRO DE RUÍDO
        if (abs(corrente) < ruidoMinimo) {
            corrente = 0;
        }

        soma += corrente * corrente;

        delayMicroseconds(200);
    }

    float rms = sqrt(soma / amostras);

    return rms;
}

// =====
// MEDIÇÃO RMS DA TENSÃO
// =====
float medirTensaoRMS() {

    int amostras = 1000;

    float soma = 0;

```

```

for (int i = 0; i < amostras; i++) {

    float leitura =
        analogRead(pinTensao) * (vRef / adcMax);

    leitura -= vRef / 2.0;

    // FILTRO DE RUÍDO
    if (abs(leitura) < ruidoMinimo) {
        leitura = 0;
    }

    soma += leitura * leitura;

    delayMicroseconds(100);
}

float vrmsSensor = sqrt(soma / amostras);

float tensaoRede =
    vrmsSensor * fatorCalibracaoTensao;

// VALIDAÇÃO
if (tensaoRede < 5) {
    tensaoRede = 0;
}

return tensaoRede;
}

// =====
// CÁLCULO DE ENERGIA
// =====
void atualizarEnergia(float potencia) {

    unsigned long agora = millis();

    float deltaTempo =
        (agora - tempoAnterior) / 1000.0;

    tempoAnterior = agora;

```

```

energia_kWh +=
    (potencia / 1000.0) *
    (deltaTempo / 3600.0);
}

// =====
// CHECKSUM SIMPLES
// =====
int gerarChecksum(
    float tensao,
    float corrente,
    float potencia
){

    int checksum =
        (int)tensao +
        (int)(corrente * 100) +
        (int)potencia;

    return checksum % 255;
}

// =====
// LOOP PRINCIPAL
// =====
void loop() {

    if (millis() - ultimoEnvio >= intervalo) {

        ultimoEnvio = millis();

        // ===== LEITURAS =====
        float corrente = medirCorrenteRMS();

        float tensao = medirTensaoRMS();

        // ===== POTÊNCIA =====
        float potencia =
            tensao * corrente;

```

```

// ===== ENERGIA =====
atualizarEnergia(potencia);

// ===== VALIDAÇÃO =====
if (corrente < 0) corrente = 0;
if (tensao < 0) tensao = 0;
if (potencia < 0) potencia = 0;

// ===== CHECKSUM =====
int checksum =
    gerarChecksum(
        tensao,
        corrente,
        potencia
    );

/*
=====
PROTOCOLO ESTRUTURADO

T:127.5;
I:0.32;
P:40.8;
E:0.0021;
C:123

=====
*/

Serial.print("T:");
Serial.print(tensao, 2);

Serial.print(";I:");
Serial.print(corrente, 3);

Serial.print(";P:");
Serial.print(potencia, 2);

Serial.print(";E:");
Serial.print(energia_kWh, 6);

```

```
    Serial.print(";C:");  
    Serial.println(checksum);  
  }  
}
```